ΨΗΦΙΑΚΕΣ ΕΠΙΚΟΙΝΩΝΙΕΣ (ECE_K510) | ΕΡΓΑΣΤΗΡΙΟ

Εισαγωγή στο Software Defined Radio & GNU Radio

Εισαγωγικό Κεφάλαιο · Εργαστήριο 0

RTL-SDR HackRF One GNU Radio 3.10 A. Κούτρας

Χάρτης χρωμάτων — τι σημαίνει κάθε χρώμα στο φυλλάδιο

- **Μπλε** — ορισμοί & θεωρία
- **Πορτοκαλί** — προσοχή / συνηθισμένα λάθη
- **Μωβ** — δραστηριότητες για εσάς
- **Χρυσό** — δικός σας κώδικας (custom blocks)
- **Πράσινο** — συμβουλές, βήματα πειράματος
- **Κόκκινο** — νομικά / εκπομπή / sinks
- **Τιρκουάζ** — αυτοέλεγχος γνώσεων
- `code` **Ροζ monospace** — όνομα αρχείου / block / παράμετρος

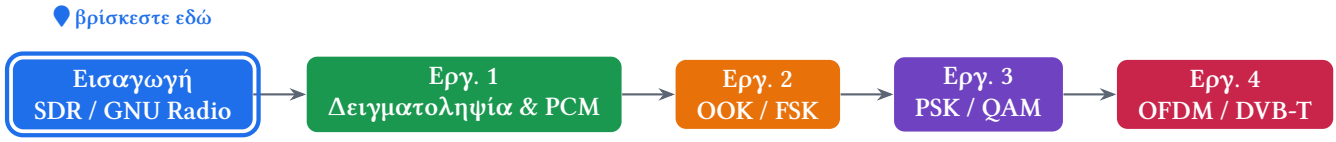
Σκοπός

Πριν μπείτε στη δειγματοληψία, τη διαμόρφωση και το OFDM των επόμενων ασκήσεων, χρειάζεστε δύο πράγματα:

(1) να καταλάβετε **τι είναι ένα SDR** και πώς διαφέρει από μια συμβατική ραδιοσυσκευή·

(2) να μάθετε το **GNU Radio**, το εργαλείο με το οποίο θα «χτίζετε» δέκτες και πομπούς σαν διαγράμματα ροής σε *όλα* τα εργαστήρια που ακολουθούν.

Στο τέλος θα έχετε εγκαταστήσει το λογισμικό, θα έχετε συνδέσει το RTL-SDR σας και θα έχετε τρέξει τα πρώτα σας flowgraphs.



Προσδοκώμενα αποτελέσματα

Όταν ολοκληρώσετε το κεφάλαιο, θα:

- εξηγείτε τι σημαίνει «Software Defined Radio» και πού βρίσκεται το όριο αναλογικού/ψηφιακού σε μια τέτοια αρχιτεκτονική·
- γνωρίζετε τα βασικά χαρακτηριστικά του **RTL-SDR** (δέκτης) και του **HackRF One** (πομπός/δέκτης)·
- έχετε εγκαταστήσει το SDR toolchain και το GNU Radio, σε **Ubuntu**, **Windows** ή **macOS**·
- αναγνωρίζετε τι εφαρμογές μπορεί να «ακούσει» ένας δέκτης SDR πέρα από το FM ραδιόφωνο·
- επιλέγετε **κεραία** για δεδομένη συχνότητα και εξηγείτε gain, directivity, πόλωση, σύνθετη αντίσταση, εύρος ζώνης·
- περιγράφετε τι είναι **block** και **flowgraph** και γνωρίζετε τις βασικές οικογένειες blocks·

- έχετε τρέξει το πρώτο flowgraph («Hello World»), συνδέσει το RTL-SDR και δει σήμα σε *χρόνο, συχνότητα και waterfall*·
- έχετε γράψει (ή τροποποιήσει) ένα δικό σας **Embedded Python Block**.

Τι θα χρειαστείτε

Υπολογιστής

Ubuntu, Windows *ή* macOS (Intel ή Apple Silicon), με δυνατότητα εγκατάστασης του **GNU Radio 3.10**.

Υλικό

RTL-SDR dongle με την κεραία του (ή οποιαδήποτε συμβατή κεραία SMA/MCX).

Αρχεία

Φάκελος `student/grc/`: `hello_world.grc`, `sdr_first_look.grc`, `custom_smoother.grc`. Για Ubuntu: `setup/SETUP.md` και `setup/check_environment.sh`.

Έννοιες κλειδιά

SDR: Software Defined Radio · ADC / DAC · RTL-SDR · HackRF One · half-duplex · sample rate · I/Q

Κεραίες: gain · directivity · radiation pattern · πόλωση · impedance / matching · bandwidth

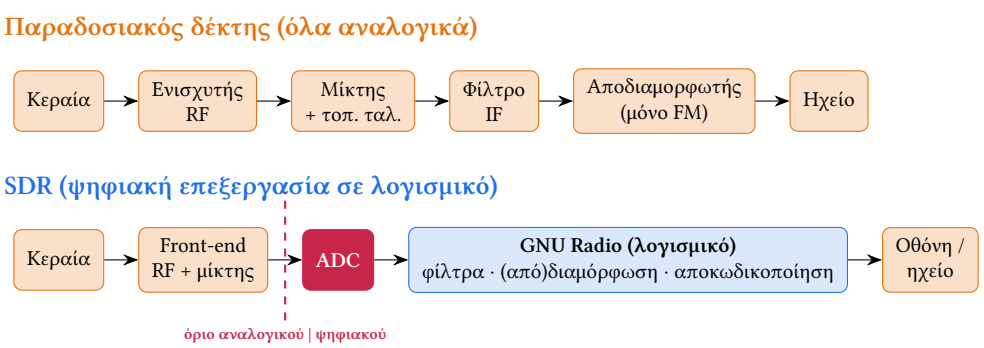
GNU Radio: GRC · flowgraph · block · stream · Throttle · Time Sink · Frequency Sink · Waterfall Sink · Embedded Python Block · work()

Εισαγωγικές Παρατηρήσεις

A. Software Defined Radio

A.1 Τι είναι ένα SDR

Σε μια **παραδοσιακή** ραδιοσυσκευή κάθε λειτουργία — ενίσχυση, μείξη (mixing), φιλτράρισμα, αποδιαμόρφωση — γίνεται με *αποκλειστικά αναλογικό κύκλωμα*: για να λάβετε FM αντί για AM χρειάζεστε άλλο κύκλωμα. Σε ένα **SDR** ο αναλογικός κόσμος περιορίζεται στο ελάχιστο: κεραία, μια αλυσίδα ενίσχυσης/μείξης και ένας **μετατροπέας** (**ADC** στη λήψη, **DAC** στην εκπομπή). Ό,τι ακολουθεί είναι **λογισμικό**: αριθμητικές πράξεις πάνω σε δείγματα.



Σχήμα 1: Σε ένα SDR το όριο αναλογικού/ψηφιακού «τραβιέται» όσο πιο κοντά στην κεραία γίνεται.

Δύο βασικά μεγέθη ενός SDR

- **Ρυθμός δειγματοληψίας (sample rate)**: καθορίζει πόσο «πλατύ» τμήμα του φάσματος βλέπετε ταυτόχρονα γύρω από την κεντρική συχνότητα (*instantaneous bandwidth* ≈ sample rate).
- **Bits του ADC/DAC**: καθορίζουν το *δυναμικό εύρος* — πόσο αδύναμο σήμα ξεχωρίζει από τον θόρυβο δίπλα σε ένα ισχυρό.

Θα τα ξαναδείτε, με αυστηρό ορισμό, στο Εργαστήριο 1 (δειγματοληψία, κβάντιση).

A.2 Ο εξοπλισμός του εργαστηρίου

	RTL-SDR (φοιτητές)	HackRF One (διδάσκων)
Ρόλος	**Μόνο λήψη (RX only)**	Λήψη **και** εκπομπή (**half-duplex**: όχι ταυτόχρονα)
Συχνότητες	≈ 24 MHz – 1.7 GHz (R820T2)	1 MHz – 6 GHz
ADC/DAC	8 bit	8 bit
Εύρος (inst. BW)	≈ 2.4 MHz (πρακτικά)	έως 20 MHz
Σύνδεσμος	συνήθως MCX	SMA
Διαθεσιμότητα	φθηνό, ένα ανά φοιτητή	ένα ανά εργαστήριο, το χειρίζεται ο διδάσκων

Το **RTL-SDR** ξεκίνησε ως τσιπάκι φθηνών TV tuner (DVB-T) που αποδείχθηκε ότι εκθέτει τα ακατέργαστα δείγματα I/Q — από εκεί προήλθε η οικογένεια των «φθηνών SDR». Είναι ιδανικό ως δέκτης φοιτητή, αλλά **δεν εκπέμπει**. Το **HackRF One** (open-source hardware) καλύπτει σχεδόν όλο το ενδιαφέρον φάσμα και εκπέμπει, γι' αυτό χρησιμοποιείται **μόνο από τον διδάσκοντα**.

Εκπομπή: μόνο από τον διδάσκοντα

Οι επιδείξεις TX γίνονται στη ζώνη ISM **433 MHz**, σε **χαμηλή ισχύ**. Η εκπομπή εκτός ζωνών ISM ή πάνω από το επιτρεπτό όριο ισχύος παραβιάζει τη νομοθεσία τηλεπικοινωνιών. **Μην εκπέμπετε ποτέ** χωρίς την καθοδήγηση του διδάσκοντα — οι φοιτητές χρησιμοποιούν το SDR τους μόνο για *λήψη*.

A.3 Τι μπορείτε να κάνετε με έναν δέκτη SDR

Ο ίδιος δέκτης, με διαφορετικό λογισμικό, «ακούει» ένα εντυπωσιακά ευρύ φάσμα σημάτων:

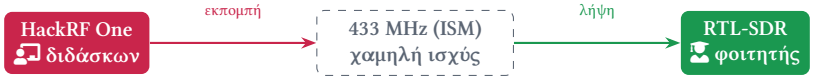
Εφαρμογή	Συχνότητα	Τι βλέπετε / εργαλείο
ADS-B	`1090 MHz`	Θέση, ύψος, ταχύτητα αεροσκαφών σε χάρτη πραγματικού χρόνου (`dump1090`).
AIS	`161.975 / 162.025 MHz`	Θέση και πορεία πλοίων.
NOAA APT	`137 MHz`	Αναλογικές εικόνες σύννεφων από πολικούς δορυφόρους (`noaa-apt`, `wxtoimg`).
Αισθητήρες ISM	`433 / 868 MHz`	Καιρικοί σταθμοί, τηλεχειριστήρια, αισθητήρες πίεσης ελαστικών (`rtl_433`).
Pagers / APRS	`VHF/UHF`	POCSAG και ραδιοερασιτεχνικά πακέτα.
Ραδιοαστρονομία	`1420.4 MHz`	Γραμμή υδρογόνου (απαιτητική διάταξη με horn).
Σαρωτής φάσματος	`ευρύ`	PMR, ασύρματα μικρόφωνα, ρεπίτερ: ορατά στο waterfall, ακόμη κι αν είναι ψηφιακά.

Νομικό / ηθικό πλαίσιο

Η **λήψη** σημάτων γενικής εκπομπής (broadcast, ADS-B, AIS, καιρικοί δορυφόροι) είναι συνήθως ελεύθερη. Η αποκωδικοποίηση/χρήση περιεχομένου ιδιωτικών ή κρυπτογραφημένων επικοινωνιών (π.χ. κινητή τηλεφωνία) υπόκειται σε νομικούς περιορισμούς. Μείνετε στις εφαρμογές του πίνακα.

A.4 Η πλευρά της εκπομπής (TX)

Ό,τι κάνει τον δέκτη SDR «universal», το κάνει και τον πομπό: το ίδιο υλικό μπορεί να παράγει σχεδόν οποιοδήποτε σήμα περιγράφεται σε λογισμικό — έναν τόνο, μια ψηφιακή διαμόρφωση OOK/FSK ή PSK/QAM, ακόμη και ολόκληρο σήμα DVB-T. Στο μάθημα η εκπομπή είναι **επίδειξη του διδάσκοντα**: μεταδίδεται ένας απλός τόνος για λίγα δευτερόλεπτα και εσείς τον βλέπετε να εμφανίζεται στο *δικό σας* waterfall. Η λογική **«TX flowgraph → ουρανός (φάσμα) → RX flowgraph»** επαναλαμβάνεται σε κάθε εργαστήριο διαμόρφωσης.



A.5 Κεραίες: χαρακτηριστικά και τύποι

Η κεραία είναι το μοναδικό πραγματικά «αναλογικό» και μη επαναπρογραμματιζόμενο κομμάτι της αλυσίδας: καθορίζει *ποιες* συχνότητες φτάνουν στον ADC με αξιοπρεπή στάθμη. Πέντε μεγέθη την περιγράφουν:

Κέρδος (gain, dBi/dBd)	Πόσο «συγκεντρώνει» την ενέργεια προς μια κατεύθυνση, σε σχέση με ιδανική ισοτροπική πηγή (dBi) ή δίπολο αναφοράς (dBd). Περισσότερο κέρδος = μεγαλύτερη εμβέλεια προς μία κατεύθυνση, λιγότερη κάλυψη προς τις άλλες.
Κατευθυντικότητα / διάγραμμα ακτινοβολίας	Πώς κατανέμεται η ευαισθησία στον χώρο: **πανκατευθυντική** (δίπολο, whip) ή **κατευθυντική** (Yagi).
Πόλωση	Προσανατολισμός του ηλεκτρικού πεδίου: **γραμμική** (κατακόρυφη/οριζόντια — πρέπει να ταιριάζει με τον πομπό, αλλιώς χάνετε στάθμη) ή **κυκλική** (άγνωστος/μεταβαλλόμενος προσανατολισμός, π.χ. δορυφόροι).
Σύνθετη αντίσταση & προσαρμογή	Το SDR υλικό έχει είσοδο/έξοδο **50 Ω**. Μια κεραία προσαρμοσμένη σε άλλη αντίσταση *ανακλά* ενέργεια αντί να την ακτινοβολεί/λαμβάνει.
Εύρος ζώνης	Το εύρος συχνοτήτων όπου η κεραία παραμένει προσαρμοσμένη. Κεραία για 433 MHz αποδίδει χειρότερα στα 100 MHz.

Τύπος	Μοτίβο	Τυπική χρήση εδώ
Whip / monopole ($\lambda/4$)	πανκατευθυντική	Η κεραία του RTL-SDR/HackRF: απλή, φθηνή, θέλει επίπεδο γείωσης.
Δίπολο ($\lambda/2$)	πανκατευθυντική (σε ένα επίπεδο)	FM και ραδιοερασιτεχνικά· καλύτερο gain από monopole.
Discone	πανκατευθυντική, ευρυζωνική	Σαρωτής φάσματος γενικής χρήσης (VHF/UHF).
Yagi-Uda	κατευθυντική, υψηλό gain	ADS-B, ασθενής σταθερός πομπός.
Log-periodic	κατευθυντική, ευρυζωνική	Μετρήσεις σε ευρύ φάσμα.
Βρόχος (loop)	πανκατευθυντική, βαθύ null	Direction finding, χαμηλές συχνότητες.
Quadrifilar helix (QFH)	σχεδόν σφαιρική, κυκλική πόλωση	Δορυφόροι NOAA.

Παράδειγμα: μήκος $\lambda/4$ για τα 433 MHz

$$\frac{\lambda}{4} = \frac{c}{4f} = \frac{3 \cdot 10^8}{4 \cdot 433 \cdot 10^6} \approx \mathbf{17.3\ cm}$$

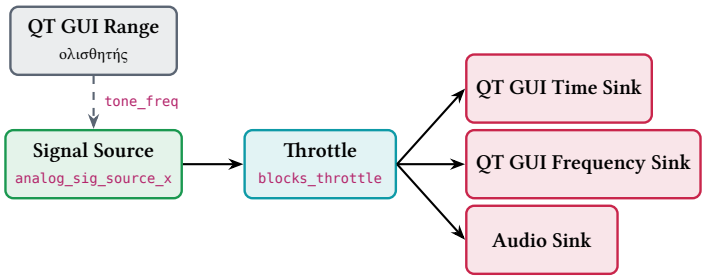
Αυτό είναι ακριβώς το μήκος των περισσότερων κεραιών «rubber duck» στα 433 MHz.

B. GNU Radio

B.1 Τι είναι το GNU Radio

Το **GNU Radio** είναι ελεύθερο toolkit για την υλοποίηση συστημάτων SDR. Δεν είναι το SDR αλλά το πλαίσιο όπου *περιγράφετε την επεξεργασία σήματος*. Ένα πρόγραμμα ονομάζεται **flowgraph**: κατευθυνόμενο γράφημα από **blocks** (αυτόνομες μονάδες επεξεργασίας) συνδεδεμένα με **streams** (σωλήνες από τους οποίους ρέουν δείγματα σε σταθερό ρυθμό). Στο παρασκήνιο κάθε block τρέχει σε δικό του thread και ο **scheduler** φροντίζει τη ροή δεδομένων.

Το **GNU Radio Companion (GRC)** είναι το γραφικό περιβάλλον: σέρνετε blocks σε καμβά, τα συνδέετε, ορίζετε παραμέτρους και το GRC **παράγει αυτόματα** τον κώδικα Python που υλοποιεί το flowgraph.



Σχήμα 2: Το flowgraph `hello_world.grc`. Το χρώμα κάθε block δείχνει την οικογένειά του (Πίνακας B.2).

Κάθε flowgraph έχει δύο «υποχρεωτικά» στοιχεία

Ένα block **Options** (εδώ πάντα `Generate Options = QT GUI`) και συνήθως μια μεταβλητή `samp_rate`. Στη γραμμή εργαλείων: **Generate** (παράγει τον κώδικα) → **Run** → **Kill/Stop**. Διπλό κλικ σε block ανοίγει τις παραμέτρους του.

B.2 Οικογένειες blocks

Οι εκατοντάδες blocks οργανώνονται σε λίγες οικογένειες. Στο φυλλάδιο η κάθε οικογένεια έχει *σταθερό χρώμα*:

Οικογένεια	Παραδείγματα	Ρόλος
ΠΗΓΕΣ	Signal Source, Noise Source, Vector/File Source, SDR Source	Παράγουν το αρχικό stream δειγμάτων
ΚΑΤΑΒΟΘΡΕΣ	Null / File / Audio Sink, SDR Sink, QT GUI Sinks	Καταναλώνουν, απεικονίζουν, αποθηκεύουν
ΜΑΘΗΜΑΤΙΚΑ	Add, Multiply, Multiply Const, Subtract	Πράξεις δείγμα-προς-δείγμα
ΦΙΛΤΡΑ	Low/High/Band Pass, FIR/IIR, Root-Raised-Cosine	Επιλογή και διαμόρφωση φάσματος
(ΑΠΟ)ΔΙΑΜΟΡΦΩΤΕΣ	WBFM/NBFM Receive, Constellation Mod/Decoder, GMSK	Πυρήνας κάθε δέκτη/πομπού
ΑΛΛΑΓΗ ΡΥΘΜΟΥ	Throttle, Rational Resampler, Keep 1 in N, Repeat	Προσαρμογή sample rate μεταξύ σταδίων
ΣΥΓΧΡΟΝΙΣΜΟΣ	Costas Loop, Polyphase Clock Sync, Correlate Access Code	Ανάκτηση φέροντος / ρολογιού / πλαισίου
ΜΕΤΑΤΡΟΠΕΙΣ ΤΥΠΟΥ	Float To Complex, Complex To Mag/Phase, Char To Float	Αλλαγή τύπου δεδομένων στη σύνδεση
CUSTOM	Embedded Python Block	Το δικό σας block (βλ. B.5)

Πού θα τα ξαναδείτε: **φίλτρα** και **αλλαγή ρυθμού** στο Εργ. 1, **διαμορφωτές** στα Εργ. 2–3, **συγχρονισμός** στο Εργ. 4 (OFDM/DVB-T).

B.3 Απεικόνιση σημάτων: χρόνος, φάσμα, waterfall

Τρία sinks καλύπτουν το 90% των αναγκών σας για «να δω τι συμβαίνει»:



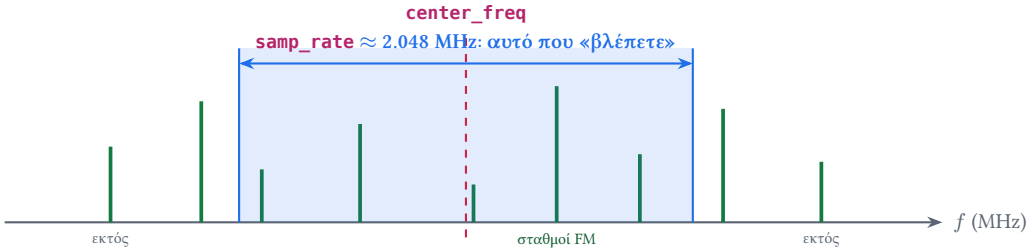
Σχήμα 3: Σχηματική όψη των τριών βασικών απεικονίσεων. Το Waterfall δείχνει και τον χρόνο: εύκολα εντοπίζονται διαλείποντα σήματα (στήλες που εμφανίζονται/χάνονται, burst).

- **QT GUI Time Sink** — πλάτος έναντι χρόνου (σαν παλμογράφος). Κυματομορφές, σκάλες κβάντισης, ροές bits. Παράμετροι: `size`, `srate` (σωστός άξονας χρόνου), trigger.
- **QT GUI Frequency Sink** — φασματική πυκνότητα ισχύος (dB) έναντι συχνότητας, μέσω FFT. Παράμετροι: `fftsize` (ανάλυση), `wintype` (παράθυρο), `average` (εξομάλυνση). Δεξί κλικ → *Max Hold* «κρατά» το ανώτερο επίπεδο ανά bin — χρήσιμο για μέτρηση εύρους ζώνης.
- **QT GUI Waterfall Sink** — ίδιος άξονας συχνότητας με το Frequency Sink, αλλά ο κατακόρυφος άξονας είναι ο *χρόνος* και η ισχύς κωδικοποιείται ως *χρώμα*. Το Frequency Sink είναι «φωτογραφία», το Waterfall «ταινία».

Αργότερα θα δείτε και το **QT GUI Constellation Sink** (I έναντι Q) στα εργαστήρια PSK/QAM.

B.4 Πηγή SDR: τι «βλέπει» το φάσμα σας

Το block `soapy_rtlsdr_source` κάνει όλη τη δουλειά με τρεις παραμέτρους: `samp_rate`, `center_freq` και `gain`. Ό,τι φτάνει στην κεραία γύρω από την `center_freq`, σε εύρος $\pm$ `samp_rate`/2, γίνεται ένα stream μιγαδικών δειγμάτων (`complex` = ζεύγη I/Q).



Σχήμα 4: Η πηγή SDR «βλέπει» ένα παράθυρο πλάτους ≈ `samp_rate` γύρω από την `center_freq`. Για να δείτε σταθμό στην άκρη, μετακινήστε τη συχνότητα συντονισμού.

⚠ Η πηγή SDR δεν θέλει Throttle

Η πηγή SDR είναι το *μοναδικό* block του flowgraph που δεν χρειάζεται Throttle: ο ρυθμός επιβάλλεται από το ίδιο το υλικό (USB). Throttle *μαζί* με SDR πηγή/καταβόθρα δίνει την προειδοποίηση «two rate limiting blocks». Στο `hello_world.grc` (χωρίς υλικό) το Throttle είναι απαραίτητο.

B.5 Δικά σας blocks σε Python

Όταν κανένα έτοιμο block δεν κάνει ό,τι θέλετε, γράφετε το δικό σας με το **Embedded Python Block** (`epy_block`): ανοίγει επεξεργαστή κώδικα μέσα στο GRC και ό,τι γράψετε γίνεται block με δικές του εισόδους/εξόδους/παραμέτρους. Παράδειγμα από το `custom_smoother.grc`:

```
import numpy as np
from gnuradio import gr


class blk(gr.sync_block):(1)
    """Moving-average smoother."""

    def __init__(self, window=5):(2)
        gr.sync_block.__init__(
            self,
            name='Smoother (moving average)',
            in_sig=[np.float32],(3)
            out_sig=[np.float32],
        )
        self.window = window(4)
        self.history = np.zeros(0, dtype=np.float32)(5)

    def work(self, input_items, output_items):(6)
        x = input_items[0]
        w = max(1, int(self.window))
        if len(self.history) != w - 1:
            self.history = np.zeros(w - 1, dtype=np.float32)
        buf = np.concatenate([self.history, x])
        cumsum = np.cumsum(np.insert(buf, 0, 0.0))
        y = (cumsum[w:] - cumsum[:-w]) / w
        self.history = buf[-(w - 1):] if w > 1 else np.zeros(0, dtype=np.float32)
        output_items[0][:] = y.astype(np.float32)
        return len(output_items[0])(7)
```

</> Οι κανόνες κάθε custom block

1. Η κλάση **πρέπει** να λέγεται `blk` και να κληρονομεί από `gr.sync_block` (1 δείγμα εισόδου → 1 εξόδου). Υπάρχουν και `gr.decim_block`, `gr.interp_block`, `gr.basic_block`.
2. Ο constructor **πρέπει** να καλείται *χωρίς ορίσματα*: κάθε παράμετρος έχει προεπιλογή.
3. `in_sig`/`out_sig` δηλώνουν τον τύπο και το πλήθος των θυρών.
4. Κάθε attribute `self.x = ...` γίνεται **ζωντανά ρυθμιζόμενη παράμετρος**: το GRC παράγει `set_x()`, ώστε να τη δέσετε σε *QT GUI Range* (εδώ το `window`).
5. Το block έχει **κατάσταση** που διατηρείται μεταξύ κλήσεων του `work()`.
6. Το `work()` καλείται επανειλημμένα από τον scheduler και παίρνει numpy arrays.
7. Επιστρέφει το πλήθος των δειγμάτων που πράγματι παρήγαγε.

⚠ Gotcha: το GNU Radio δίνει δεδομένα σε «κομμάτια»

Το `work()` παίρνει buffers *αυθαίρετου μεγέθους*, όχι ολόκληρο το σήμα. Γι' αυτό το block κρατά τα τελευταία $w - 1$ δείγματα στο `self.history`: χωρίς αυτά θα εμφανιζόταν παραμόρφωση στην αρχή κάθε buffer.

Στο `custom_smoother.grc` ένα ημίτονο προστίθεται σε λευκό θόρυβο Gauss και περνά από το custom block. Το Time Sink δείχνει *και τα δύο* σήματα. Μεγαλύτερο παράθυρο ⇒ καθαρότερο σήμα, αλλά πιο «θολή» και καθυστερημένη απόκριση σε απότομες μεταβολές (*trade-off*).

Πείραμα

Πώς να δουλέψετε

Ακολουθήστε τα 1 2 ... με τη σειρά. Μόνο το Βήμα 3 και το 4 χρειάζονται το RTL-SDR. Το Βήμα 2 (`hello_world`) τρέχει σε οποιονδήποτε υπολογιστή.

Βήμα 1 — Εγκατάσταση λογισμικού

Χρειάζεστε GNU Radio 3.10, τον οδηγό SDR (`gr-osmosdr` ή/και `gr-soapy`) και τα εργαλεία `rtl-sdr`/`hackrf`.

Ubuntu

```
sudo apt update
sudo apt install gnuradio \
  gr-osmosdr rtl-sdr hackrf
```

Το Linux αναγνωρίζει το RTL-SDR ως συσκευή **DVB-T** και το «αρπάζει» το άρθρωμα `dvb_usb_rtl28xxu`. Πρέπει να μπει σε **blacklist**. Τα πλήρη βήματα (και το ζήτημα με Anaconda) στο `setup/SETUP.md`. Επιβεβαίωση:

```
./setup/check_environment.sh
```

Ελέγχει GRC, module οδηγού, `rtl_test`, `hackrf_info`. Χωρίς συνδεδεμένη συσκευή, τα `[WARN]` είναι αναμενόμενα.

Windows

Δεν υπάρχει `apt`: χρησιμοποιήστε την αυτόνομη διανομή radioconda.

1. Κατεβάστε και εκτελέστε τον installer της radioconda (Windows x86-64).
2. Ανοίξτε το **GNU Radio Companion** από το μενού Έναρξης.
3. **RTL-SDR**: αντικαταστήστε τον driver των Windows με **WinUSB** μέσω του Zadig (*Options* → *List All Devices* αν δεν φαίνεται).
4. Επαληθεύστε: ανοίξτε το `sdr_first_look.grc` και πατήστε **Run**.

macOS (Intel και Apple Silicon)

Όπως στα Windows, η πιο απλή διαδρομή είναι η αυτόνομη διανομή radioconda, που περιλαμβάνει GNU Radio, οδηγούς SDR και εξαρτήσεις. Στο macOS **δεν** χρειάζεται blacklist (Ubuntu) ούτε Zadig (Windows).

1. Από τη σελίδα releases της radioconda κατεβάστε τον installer `.sh` που ταιριάζει στον Mac σας: **arm64** για Apple Silicon (M1/M2/...) ή **x86_64** για Intel. (Μενού Apple → «Σχετικά με αυτό το Mac»: δείτε το πεδίο «Chip»/«Επεξεργαστής».)
2. Ανοίξτε το **Terminal** και τρέξτε τον installer, αποδεχόμενοι τις προεπιλογές:
```
bash ~/Downloads/radioconda-*-MacOSX-*.sh
```
3. Κλείστε και ανοίξτε ξανά το Terminal, ώστε να ενεργοποιηθεί το environment, και ξεκινήστε το GRC:
```
gnuradio-companion
```
4. Συνδέστε το RTL-SDR και ελέγξτε το με `rtl_test -t`. Επαληθεύστε και με το `sdr_first_look.grc` (→ **Run**).

macOS: συνηθισμένα προβλήματα

- Αν το macOS μπλοκάρει κάποιο εκτελέσιμο ως «από μη επαληθευμένο προγραμματιστή»: *System Settings → Privacy & Security → Open Anyway.*
- Το `./setup/check_environment.sh` γράφτηκε για Ubuntu: σε Mac ελέγξτε χειροκίνητα με `gnuradio-companion --version` και `rtl_test -t`.
- Χρησιμοποιήστε απευθείας θύρα USB του Mac (όχι hub χωρίς τροφοδοσία): το RTL-SDR τραβά αρκετό ρεύμα.

Ubuntu: αν το GNU Radio δεν «βλέπει» το dongle

Το πιο συνηθισμένο αίτιο είναι το φορτωμένο `dvb_usb_rtl28xxu` ή ένα ενεργό conda environment. Τρέξτε `rtl_test -t`: πρέπει να αναγνωρίσει τη συσκευή και να αναφέρει τον ρυθμό δειγματοληψίας που υποστηρίζει.

Βήμα 2 — Το πρώτο μου flowgraph: «Hello World»

Αρχείο: `student/grc/hello_world.grc` (δεν χρειάζεται SDR)

1. Ανοίξτε το αρχείο στο GRC και πατήστε **Run**.
2. Μετακινήστε τους δύο ολισθητές **QT GUI Range**: η συχνότητα (`tone_freq`, 100–2000 Hz) και το πλάτος (`tone_amp`) αλλάζουν *ζωντανά*, χωρίς να σταματήσει το flowgraph.
3. Παρατηρήστε το ημίτονο στο **Time Sink**, τη φασματική κορυφή στο **Frequency Sink** και ακούστε το από το **Audio Sink**.

Τι κάνει κάθε block: Signal Source παράγει το ημίτονο· Throttle προσομοιώνει τον πραγματικό ρυθμό `samp_rate` (χωρίς υλικό τίποτα δεν θα «επιβράδυνε» το πρόγραμμα)· Sinks απεικονίζουν/αναπαράγουν. Ο σκελετός *πηγή → throttle → οπτικοποίηση* είναι ο σκελετός **κάθε** flowgraph προσομοίωσης των επόμενων εργαστηρίων.

Βήμα 3 — Πρώτη λήψη με το RTL-SDR

Αρχείο: `student/grc/sdr_first_look.grc`

1. Συνδέστε το RTL-SDR και βιδώστε την κεραία. Σε Ubuntu/macOS τρέξτε `rtl_test -t`.

2. Ανοίξτε το `sdr_first_look.grc` και πατήστε **Run**.
3. Μετακινήστε τον ολισθητή **«Συχνότητα συντονισμού (MHz)»** γύρω στα **100 MHz** (τοπική ζώνη FM). Στο Frequency Sink εμφανίζονται οι σταθμοί ως *κορυφές*, στο Waterfall ως *κάθετες λωρίδες*.
4. Δοκιμάστε τον ολισθητή **«Κέρδος RF (dB)»**: πολύ μικρό ⇒ χάνετε ασθενείς σταθμούς, πολύ μεγάλο ⇒ ο δέκτης μπαίνει σε κορεσμό και εμφανίζονται «φαντάσματα».

Η πηγή (`soapy_rtlsdr_source`) τροφοδοτεί **τρία** sinks σε καρτέλες: Frequency Sink (φάσμα RF), Waterfall Sink και Time Sink (οι ακατέργαστες κυματομορφές I/Q).

Κάτι έτοιμο για να «ακούσετε» σε 2 λεπτά

Αν θέλετε ένα έτοιμο εργαλείο χωρίς flowgraph, δοκιμάστε το GQRX (Ubuntu/macOS) ή το **SDR#** (Windows): επιλέξτε τον οδηγό RTL-SDR και συντονιστείτε σε σταθμό FM.

Gotcha: το Osmocom Source πετάει σφάλμα

Αν το block `osmocom_source` (από το `gr-osmosdr`) δίνει σφάλμα στην εκκίνηση αντί να λαμβάνει σήμα, δοκιμάστε το αντίστοιχο της Soapy (`soapy_rtlsdr_source`, όπως εδώ). Καλύπτουν το ίδιο υλικό μέσω διαφορετικής βιβλιοθήκης και σε ορισμένες εγκαταστάσεις δουλεύει αξιόπιστα μόνο η μία.

Βήμα 4 — Το δικό σας block

Αρχείο: `student/grc/custom_smoother.grc`

1. Τρέξτε το flowgraph. Στο Time Sink βλέπετε το θορυβώδες και το εξομαλυμένο σήμα μαζί.
2. Μετακινήστε το **«Πλάτος θορύβου»** και το **«Παράθυρο εξομάλυνσης»** και παρατηρήστε το trade-off καθαρότητας / καθυστέρησης.
3. Διπλό κλικ στο `epy_block_0` ανοίγει τον κώδικα (Δραστηριότητα 5).

Δραστηριότητες

Δραστηριότητα 1 — Εγκατάσταση και επιβεβαίωση

Εγκαταστήστε το GNU Radio 3.10 (Ubuntu, Windows ή macOS) και επιβεβαιώστε την εγκατάσταση (`./setup/check_environment.sh` σε Ubuntu, `rtl_test -t` σε macOS, ή άνοιγμα flowgraph σε Windows).

Δραστηριότητα 2 — Hello World και αρμονικές

Τρέξτε το `hello_world.grc` και αλλάξτε ζωντανά συχνότητα/πλάτος. Έπειτα αλλάξτε το `waveform` του Signal Source από ημίτονο σε τετραγωνικό (`analog.GR_SQR_WAVE`) και περιγράψτε τι αλλάζει στο *φάσμα* (αρμονικές).

Δραστηριότητα 3 — Τρεις σταθμοί FM

Με το `sdr_first_look.grc` εντοπίστε τρεις τοπικούς σταθμούς FM και συμπληρώστε:

#	Συχνότητα (MHz)	Στάθμη στο φάσμα (dB)	Σχόλιο
1			
2			
3			

Δραστηριότητα 4 — Η ζώνη 433 MHz

Συντονιστείτε στα **433 MHz** (χωρίς ενεργή εκπομπή) και περιγράψτε τι βλέπετε στο waterfall σε ήσυχη έναντι θορυβώδους περιοχής. *Υπόδειξη: δοκιμάστε να πατήσετε ένα τηλεχειριστήριο κοντά στο dongle.*

Δραστηριότητα 5 — Τροποποίηση του custom block

Πειραματιστείτε με το «Παράθυρο εξομάλυνσης» και το «Πλάτος θορύβου». Έπειτα **τροποποιήστε** το `epy_block_0`: αλλάξτε τον κινητό μέσο σε **εκθετικό εξομαλυντή**

$$y[n] \;=\; \alpha\, x[n] + (1-\alpha)\, y[n-1],$$

με το α ως νέα, ζωντανά ρυθμιζόμενη παράμετρο, και συγκρίνετε οπτικά τη συμπεριφορά με τον κινητό μέσο.

Δραστηριότητα 6 — Προαιρετικό: εφαρμογές πέρα από το FM

Αν έχετε πρόσβαση σε `dump1090` ή `rtl_433`, δοκιμάστε μία εφαρμογή του πίνακα Α.3, π.χ. ADS-B ή αποκωδικοποίηση ασύρματου αισθητήρα ISM.

Λίστα ελέγχου γνώσεων

Πριν προχωρήσετε στο Εργαστήριο 1, βεβαιωθείτε ότι μπορείτε να απαντήσετε:

- ☐ Τι κάνει ένα SDR διαφορετικό από μια παραδοσιακή ραδιοσυσκευή και πού βρίσκεται το όριο αναλογικού/ψηφιακού (ADC/DAC);
- ☐ Ποια η βασική διαφορά RTL-SDR ↔ HackRF One (εύρος συχνοτήτων, RX-only έναντι half-duplex);
- ☐ Πώς εγκαθιστάτε και επιβεβαιώνετε το GNU Radio toolchain, σε Ubuntu, Windows και macOS;
- ☐ Ονομάστε τρεις εφαρμογές λήψης SDR πέρα από το FM ραδιόφωνο.
- ☐ Τι σημαίνουν κέρδος, κατευθυντικότητα, πόλωση, σύνθετη αντίσταση και εύρος ζώνης και γιατί μια $\lambda/4$ κεραία για τα 433 MHz έχει μήκος ≈ 17 cm;
- ☐ Τι είναι **block** και **flowgraph** και ποιος ο ρόλος του Throttle σε προσομοίωση χωρίς υλικό;
- ☐ Ονομάστε τις κύριες οικογένειες blocks (πηγές, καταβόθρες, φίλτρα, (από)διαμορφωτές, αλλαγή ρυθμού, συγχρονισμός) και έναν ρόλο για καθεμία.
- ☐ Ποια η διαφορά ανάμεσα σε Time, Frequency και Waterfall Sink και πότε προτιμάτε το καθένα;
- ☐ Ποιες παράμετροι ελέγχουν μια πηγή SDR και τι εύρος φάσματος «βλέπετε» ταυτόχρονα σε σχέση με το `samp_rate`;
- ☐ Ποιοι είναι οι κανόνες κατασκευής ενός Embedded Python Block (`class blk`, `__init__` χωρίς υποχρεωτικά ορίσματα, `in_sig`/`out_sig`, `work()`) και πώς γίνεται μια παράμετρος ζωντανά ρυθμιζόμενη;

🚀 **Όλα τα παραπάνω τα έχετε**; Είστε έτοιμοι για το **Εργαστήριο 1: Δειγματοληψία & PCM**.